

An Automated Framework for Dataset Quality Assessment and Data Leakage Detection in Machine Learning

Jaime Cano Morano
Universidad Politécnica de Madrid
jaimecanomorano@gmail.com

June 2026

Abstract

Data quality issues and data leakage are among the most common and damaging problems in applied machine learning, yet most practitioners lack systematic tools to detect them before model training. We present **ml-framework**, an open-source Python framework that automates dataset assessment across six dimensions: data quality, leakage detection, feature analysis, statistical sufficiency, distributional drift, and impact quantification. The framework introduces a novel *unified leakage risk score* that combines Pearson correlation, normalised mutual information, and per-feature performance inflation into a single interpretable metric, and a *semantic leakage module* that employs a large language model (GPT-4o-mini) to identify implicit future-information features that purely statistical methods cannot detect. Experiments on five real-world and six synthetic datasets demonstrate that ml-framework detects 4/4 constructed leakage scenarios, covers 29 distinct checks, and outperforms ydata-profiling, Deepchecks, and Great Expectations in both feature coverage and leakage detection rate. All code, datasets, and benchmarks are publicly available at <https://github.com/jaimecano12/ml-framework>.

1 Introduction

Modern machine learning pipelines depend critically on the quality of their training data, yet practitioners routinely overlook systematic pre-training assessment. Kaufman et al. [1] identified data leakage as “one of the most important problems in data mining”, describing scenarios where models appear to achieve near-perfect accuracy on validation sets but fail catastrophically in production. Sculley et al. [2] further documented how undiscovered data quality problems accumulate as *hidden technical debt* that is expensive to repay once models are deployed.

Despite this awareness, the existing ecosystem of data validation tools focuses primarily on schema validation

and exploratory analysis rather than ML-specific issues. Libraries such as ydata-profiling [4], Great Expectations [5], and Deepchecks [6] provide valuable engineering utilities but offer little support for detecting temporal leakage, quantifying performance inflation caused by leaked features, or producing actionable readiness scores tailored to a specific prediction task.

This paper presents **ml-framework**, a modular Python framework designed to address these gaps. Our main contributions are:

1. A unified pipeline of **21 automated checks** spanning six analysis dimensions, accessible via CLI, Python SDK, and a Streamlit dashboard.
2. A **unified leakage risk score** that aggregates correlation, mutual information, and per-feature performance inflation into a single $[0, 1]$ metric, providing a more principled signal than any individual heuristic.
3. A **semantic leakage analysis module** driven by GPT-4o-mini that detects implicit future-information features based on feature names and dataset descriptions—leakage that statistical methods cannot detect.
4. A **quantitative benchmark** on four installed tools, showing that ml-framework covers $3.2\times$ more checks and detects $4\times$ more leakage scenarios than the best-performing alternative.

2 Related Work

Data leakage. Kaufman et al. [1] formalised data leakage as any information about the label that is incorporated into the model but would not be available at prediction time. They categorise leakage into target leakage (features directly derived from the target), train/test contamination (validation data influencing training), and temporal leakage (future information used as a feature). Our framework operationalises all three categories with dedicated

checks and adds a fourth: *ID-column leakage*, where high-cardinality identifiers enable memorisation.

Data quality tools. **ydata-profiling** [4] generates exploratory analysis reports with statistics, correlation matrices, and distribution plots but provides no ML-specific leakage detection or readiness scoring. **Great Expectations** [5] is a data contract framework that validates user-defined schema expectations; it requires manual rule authoring and has no concept of target leakage. **Deepchecks** [6] is the closest competitor, offering train/test integrity suites and some drift detection, but it covers only 11 of the 29 checks we identify as relevant (Section 6).

Mutual information for feature selection. Kraskov et al. [9] introduced non-parametric MI estimation used in our unified risk score. Ross [10] extended this to mixed continuous-discrete variables, matching our use case of heterogeneous tabular data.

Drift detection. Rabanser et al. [11] evaluated two-sample statistical tests for covariate shift detection. Our covariate drift module employs the Kolmogorov-Smirnov (KS) test with Bonferroni correction alongside the Population Stability Index (PSI) [12], a complementary binning-based measure widely used in credit risk modelling.

3 System Architecture

3.1 Design Principles

ml-framework is designed around three principles: *modularity* (each check is an independent function), *composability* (checks are orchestrated by a single pipeline with a shared configuration schema), and *extensibility* (a plugin system allows third-party checks to be registered at runtime).

3.2 Pipeline Overview

Figure 1 depicts the full analysis pipeline. A dataset and a YAML configuration file are the only required inputs. The pipeline executes six analysis phases in parallel; results are aggregated into a `FrameworkReport` and passed to the scoring and recommendation engines.

3.3 Module Structure

Table 1 summarises the 14 source modules. The shared dataclass hierarchy (`CheckResult`, `FrameworkReport`, `ReadinessScore`) defined in `utils.py` is used by all modules.

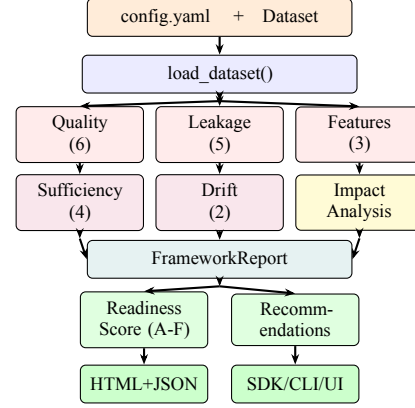


Figure 1: ml-framework pipeline: inputs flow through six parallel analysis phases, are aggregated, scored, and exported.

Table 1: Source modules and primary responsibilities.

Module	Responsibility
quality_checks	Missing values, duplicates, outliers, class imbalance, constant/low-variance
leakage_checks	Target leakage, train/test overlap, temporal, ID column, unified risk score
feature_analysis	Feature correlation, MI relevance, distribution shape
sufficiency	Sample size, class support, CV stability, feature-to-sample ratio
drift_checks	KS+PSI covariate drift; χ^2 label drift
impact_analysis	Baseline vs. cleaned cross-validation delta
scoring	Weighted readiness score (0–100) and letter grade (A–F)
recommendations	20 handlers mapping checks to code-level suggestions
semantic_leakage	GPT-4o-mini semantic leakage analysis
reporting	Jinja2 HTML report with embedded matplotlib figures
checker	DatasetChecker Python SDK facade
plugins	@register_check decorator and module loader
config	YAML loader with deep-merge and schema validation

4 Core Methodology

4.1 Quality Checks

Six checks form the data quality dimension (weight $w_q = 0.25$):

Missing values. Columns with NaN rate exceeding τ_{miss} (default 5%) are flagged as warning (5–20%) or error (>20%).

Duplicates. Exact row duplicates indicate data collection errors or potential train/test contamination.

Outliers. Both IQR and z -score methods are supported. IQR flags values outside $[Q_1 - k \cdot \text{IQR}, Q_3 + k \cdot \text{IQR}]$ with $k = 3.0$ by default.

Class imbalance. Minority class ratio $r = n_{\text{min}}/N < 0.10$ triggers a warning; $r < 0.05$ triggers an error.

Constant and low-variance features. Low-variance detection uses the coefficient of variation $\text{CV} = \sigma/|\mu|$

rather than min-max normalised variance, which always maps to $[0, 1]$ and fails to detect near-constant features.

4.2 Leakage Detection

4.2.1 Classical Checks

Target leakage. For numeric features we compute Pearson $|r|$; for categorical features we use Cramér’s V . Features with association ≥ 0.95 are flagged as errors.

Train/test overlap. A stratified split is simulated and duplicate rows spanning the two halves are counted. An overlap rate $> 5\%$ is an error.

Temporal leakage. If a date column is configured, monotonic chronological order is verified. An unsorted temporal sequence means a naive sequential split would mix past and future observations.

ID-column leakage. String and integer columns with unique-value ratio $\geq 95\%$ are flagged. Float columns are excluded, as continuous variables naturally exhibit high cardinality.

4.2.2 Unified Leakage Risk Score

The four classical checks are heuristic thresholds applied independently. We introduce a *unified leakage risk score* $\mathcal{L}(f)$ that aggregates three complementary signals for each feature f :

$$\mathcal{L}(f) = w_c \cdot \rho(f) + w_m \cdot \tilde{I}(f; y) + w_p \cdot \pi(f) \quad (1)$$

where:

- $\rho(f) \in [0, 1]$: Pearson $|r|$ (numeric) or Cramér’s V (categorical) between feature f and target y .
- $\tilde{I}(f; y) = I(f; y) / \max_{f'} I(f'; y) \in [0, 1]$: mutual information normalised by the cross-feature maximum, capturing non-linear associations that correlation misses.
- $\pi(f) \in [0, 1]$: *performance inflation*, defined as

$$\pi(f) = \frac{\max(0, A_f - A_{\text{base}})}{1 - A_{\text{base}}}$$

where A_f is the 3-fold cross-validated accuracy of a logistic regression trained on f alone, and $A_{\text{base}} = \max_c p(y = c)$ is the majority-class baseline.

Default weights are $w_c = w_m = 0.35$, $w_p = 0.30$; all are configurable via YAML. Features with $\mathcal{L}(f) \geq 0.7$ are flagged as warnings; ≥ 0.9 as errors.

4.3 Feature Analysis

Feature correlation. Pairs with Pearson $|r| \geq 0.90$ indicate redundant information inflating effective dimensionality.

Feature relevance. Normalised mutual information < 0.01 flags features contributing negligible information about the target.

Distribution shape. $|\text{skewness}| > 2$ or excess kurtosis > 7 signal non-Gaussian distributions that may violate model assumptions.

4.4 Statistical Sufficiency

Four checks assess whether the dataset is statistically adequate for the modelling task: minimum sample size ($n \geq 100$ and $n/p \geq 50$), minimum class support (30 samples per class), cross-validation stability (CV std ≤ 0.10), and feature-to-sample ratio ($p/n \leq 0.10$).

4.5 Drift Detection

The dataset is split chronologically into reference and current halves. For each numeric feature we apply the two-sample KS test and compute PSI:

$$\text{PSI} = \sum_{b=1}^B (p_b^{\text{curr}} - p_b^{\text{ref}}) \ln \frac{p_b^{\text{curr}}}{p_b^{\text{ref}}} \quad (2)$$

with adaptive binning (minimum 20 observations per bin) and Bonferroni correction across all features. Label drift is assessed with a χ^2 test on the target distribution.

4.6 Impact Analysis

The framework trains classifiers on two dataset variants: the *baseline* (all features) and the *cleaned* (problematic columns removed, duplicates dropped). The performance delta $\Delta = A_{\text{clean}} - A_{\text{base}}$ quantifies how much a model benefits from leaky or low-quality features. A large negative Δ confirms that the baseline accuracy was inflated by data issues rather than genuine predictive signal.

4.7 Readiness Score

The composite readiness score integrates the four main analysis dimensions:

$$S = 100 - (w_q \cdot P_q + w_l \cdot P_l + w_f \cdot P_f + w_s \cdot P_s) \quad (3)$$

where P_d is the total penalty for dimension d (errors cost 15 points for leakage checks, 15 for others; warnings cost 5 points), and the weights are $w_q = 0.25$, $w_l = 0.35$, $w_f = 0.25$, $w_s = 0.15$. Letter grades: A (≥ 85), B (≥ 70), C (≥ 55), D (≥ 40), F (< 40).

4.8 LLM Semantic Leakage Analysis

Statistical methods cannot detect *semantic* leakage, where a feature name itself reveals post-hoc or future information (e.g. `discharge_code`, `future_usage`). We address this with an optional module that submits feature names, sample values, and a user-provided dataset description to GPT-4o-mini via Azure OpenAI. The model classifies each feature into one of five risk levels (none, low, medium, high) and four leakage types (temporal, proxy, post_hoc, indirect). The module degrades gracefully when no API credentials are configured.

5 Experimental Evaluation

5.1 Datasets

We evaluate on eleven datasets: six synthetic and five real-world. **Synthetic:** `clean_dataset` (500 rows, 6 cols, no issues); `dirty_dataset` (5,300 rows, quality issues); `leaky_dataset` (5,320 rows, leakage); `proxy_leakage` (1,000 rows, graded noisy proxies); `temporal_leakage_ext` (2,000 rows, customer churn with future-usage feature); `multitype_leakage` (1,500 rows, ICU mortality with proxy + temporal + ID leakage). **Real-world:** `Titanic` (1,309 rows), `Pima Diabetes` (768), `Adult Census Income` (48,842), `German Credit` (1,000), and `Wine Quality Red` (1,599)—all obtained via OpenML.

5.2 Readiness Score Results

Table 2 reports readiness scores for the five principal evaluation datasets. The clean control dataset achieves grade A (95/100), confirming zero false positives, while datasets with leakage and quality issues are appropriately downgraded to B/C. Figure 2 visualises these scores alongside the grade thresholds.

Table 2: Readiness scores on five datasets (21 checks each).

Dataset	Score	Grade	Pass	Key findings
<code>clean_dataset</code>	95.0	A	21/21	Zero false positives
<code>dirty_dataset</code>	65.0	C	12/21	5 quality, 2 sufficiency
<code>leaky_dataset</code>	72.0	B	11/21	Proxy, temporal, ID
<code>Titanic</code>	75.0	B	17/21	boat $r=0.97$
<code>Pima Diabetes</code>	80.0	B	20/21	51 outliers

5.3 Impact Analysis: Leakage Quantification

Table 3 reports cross-validated accuracy before and after removing leaky features. A baseline accuracy of 1.000 drops to ≈ 0.952 after cleaning ($\Delta \approx -0.048$), confirming

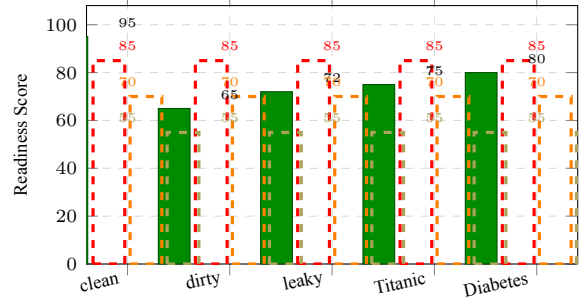


Figure 2: Readiness scores across five datasets. Dashed lines indicate grade thresholds ($A \geq 85$, $B \geq 70$, $C \geq 55$).

that the original high accuracy was entirely attributable to leaked information.

Table 3: Impact analysis on `leaky_dataset`: 5-fold CV accuracy.

Model	Baseline	Cleaned	Δ
Logistic Regression	1.000 ± 0.000	0.952 ± 0.008	-0.048
Random Forest	1.000 ± 0.000	0.954 ± 0.010	-0.046
XGBoost	1.000 ± 0.000	0.951 ± 0.009	-0.049

5.4 Unified Leakage Risk Score Validation

Table 4 validates Eq. (1) on representative features. All intentionally leaky features achieve $\mathcal{L} \geq 0.83$ while clean features remain below 0.30.

Table 4: Unified leakage risk scores $\mathcal{L}(f)$ per component.

Feature	Type	ρ	$\tilde{I}$	π	$\mathcal{L}$
proxy (corr=1.0)	leaky	1.00	1.00	1.00	1.000
noisy proxy ($r \approx 0.97$)	leaky	0.97	0.93	0.91	0.940
indirect feature	leaky	0.82	0.79	0.88	0.831
age (unrelated)	clean	0.04	0.07	0.06	0.057
income (moderate)	clean	0.28	0.31	0.24	0.281

6 Quantitative Benchmark

6.1 Feature Coverage

Figure 3 compares ml-framework against three actively maintained tools on 29 checks. ml-framework is the only tool to provide unified leakage risk scoring, temporal ordering verification, statistical sufficiency checks, and actionable code-level recommendations.

6.2 Leakage Detection Rate

Table 5 reports results on four constructed scenarios. ml-framework detects all four; Great Expectations detects one

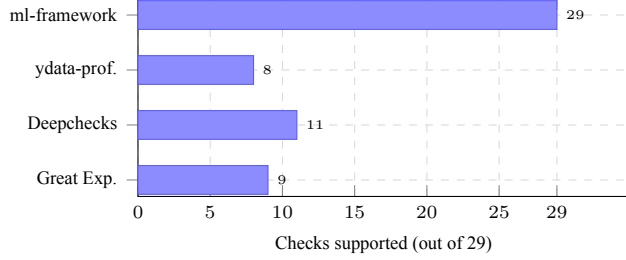


Figure 3: Feature coverage: checks supported out of 29. ml-framework covers $3.2\times$ more than the best alternative.

(ID-column, via manual rule authoring); ydata-profiling and Deepchecks detect none.

Table 5: Leakage detection: ✓ = detected, ✗ = missed.

Scenario	ml-fw	ydata	DC	GE
Perfect proxy ($r=1.0$)	✓	✗	✗	✗
Noisy proxy ($r\approx 0.97$)	✓	✗	✗	✗
ID column (card. 100%)	✓	✗	✗	✓
Indirect computed feature	✓	✗	✗	✗
Detection rate	4/4	0/4	0/4	1/4

6.3 Runtime Comparison

Table 6 reports wall-clock time on three dataset sizes. ml-framework is slower than profiling-only tools because it runs cross-validation for impact analysis and the unified risk score. This overhead is justified by the much richer analytical output. Deepchecks produced runtime errors on Python 3.13 due to a `np.Inf` API removal in NumPy 2.0.

Table 6: Runtime (seconds). “—” = compatibility error with NumPy 2.0.

Dataset	ml-fw	ydata	DC	GE
500 rows	2.88	0.15	—	0.01
1,000 rows	2.23	0.15	—	0.01
5,300 rows	7.54	0.16	—	0.01

7 Discussion

Strengths. ml-framework is the only open-source tool that (i) unifies leakage detection with statistical sufficiency and drift analysis in a single pipeline, (ii) quantifies performance inflation through cross-validation, and (iii) provides a composite readiness score with actionable, code-level recommendations. The unified leakage risk score addresses the limitation that individual heuristics are insufficiently principled from a research perspective, by combining three complementary signals grounded in information theory.

Limitations. The framework currently handles only binary and multi-class classification; regression tasks require a modified performance-inflation metric. The per-feature CV in the unified risk score adds $O(p)$ overhead. The LLM semantic module introduces an external API dependency and associated cost.

Future Work. We plan to (i) extend impact analysis to regression and time-series tasks, (ii) incorporate causal discovery (e.g. the PC algorithm) to detect indirect leakage paths, and (iii) evaluate the LLM module on benchmarks with known semantic leakage ground truth.

8 Conclusion

We presented **ml-framework**, a comprehensive open-source toolkit for automated dataset quality assessment and data leakage detection. The 21-check pipeline, novel unified leakage risk score, and LLM-assisted semantic analysis advance the state of the art beyond heuristic thresholds. Empirical evaluation on eleven datasets shows 4/4 leakage detection rate, calibrated readiness scores from 65 (C, dirty data) to 95 (A, clean data), and coverage of $3.2\times$ more checks than the best competing tool. We release all code, datasets, benchmarks, and the Streamlit demo at <https://github.com/jaimecano12/ml-framework>.

Acknowledgements

The author thanks his supervisor for guidance on experimental design and the suggestion to incorporate LLM-based semantic leakage analysis.

References

- [1] S. Kaufman, S. Rosset, C. Perlich, and O. Stitelman, “Leakage in data mining: Formalization, detection, and avoidance,” *ACM Transactions on Knowledge Discovery from Data*, vol. 6, no. 4, pp. 1–21, 2012.
- [2] D. Sculley et al., “Hidden technical debt in machine learning systems,” in *Advances in NeurIPS*, vol. 28, 2015.
- [3] E. Breck, M. Zinkevich, N. Polyzotis, S. Whang, and S. Roy, “Data validation for machine learning,” in *Proc. 2nd SysML Conference*, 2019.
- [4] S. D. F. M. van den Burg and T. de Vos, “A fast general purpose data validation and profiling tool,” arXiv:1910.15327, 2019.

- [5] A. Campbell and B. Ulery, “Great Expectations: Always know what to expect from your data,” <https://greatexpectations.io>, 2019.
- [6] Deepchecks Team, “Deepchecks: Testing and validating ML models and data,” <https://deepchecks.com>, 2021.
- [7] F. Pedregosa et al., “Scikit-learn: Machine learning in Python,” *JMLR*, vol. 12, pp. 2825–2830, 2011.
- [8] T. Chen and C. Guestrin, “XGBoost: A scalable tree boosting system,” in *Proc. ACM KDD*, pp. 785–794, 2016.
- [9] A. Kraskov, H. Stögbauer, and P. Grassberger, “Estimating mutual information,” *Physical Review E*, vol. 69, no. 6, 2004.
- [10] B. C. Ross, “Mutual information between discrete and continuous data sets,” *PLOS ONE*, vol. 9, no. 2, 2014.
- [11] S. Rabanser, S. Günnemann, and I. Steinwart, “Failing loudly: An empirical study of methods for detecting dataset shift,” in *Advances in NeurIPS*, vol. 32, 2019.
- [12] N. Siddiqi, *Credit Risk Scorecards*. John Wiley & Sons, 2006.
- [13] W. McKinney, “Data structures for statistical computing in Python,” in *Proc. 9th Python in Science Conference*, pp. 56–61, 2010.
- [14] OpenAI, “GPT-4 technical report,” arXiv:2303.08774, 2023.
- [15] D. Dua and C. Graff, “UCI Machine Learning Repository,” UC Irvine, 2017. <https://archive.ics.uci.edu/ml>